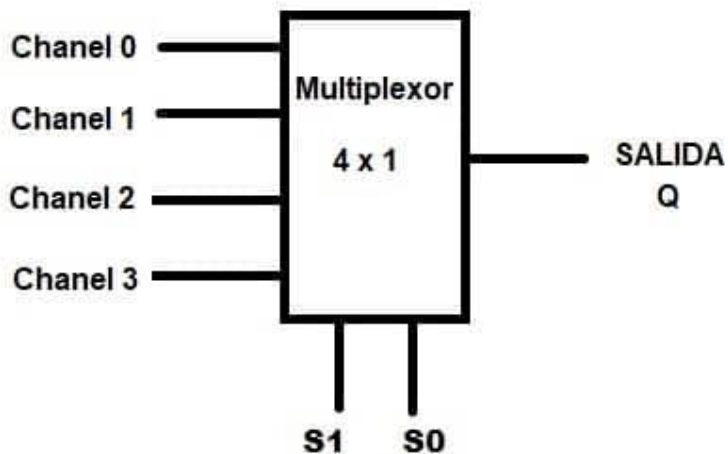
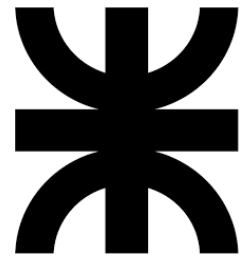




UT4 - Ejercicios de multiplexores

- **Autor:**
 - Marcos Raúl Gatica - Leg. 402006
- **Curso:** 3R2
- **Docentes:**
 - Ing. Sergio Daniel Olmedo
 - Ing. Jorge Mercado
- **Asignatura:** Técnicas Digitales I
- **Institución:** Universidad Tecnológica Nacional - Facultad Regional de Córdoba.



U
T
N

F
R
C

Índice

1. INTRODUCCIÓN	1
2. RESPUESTAS	2
2.1. Ejercicio 1	2
2.1.1. F_a	2
2.1.2. F_b	5
2.2. Ejercicio 2	8
2.2.1. f_a	8
2.2.2. f_b	9
2.2.3. f_c	10
2.3. Ejercicio 3	11

1. INTRODUCCIÓN

■ Ejercicio 1

Resolver las siguientes funciones lógicas con dos multiplexores de 4:1. Junto a cada función a resolver se encuentra la referencia a la combinación de multiplexores a utilizar. Ver la tabla de verdad para respetar las salidas correspondientes para tipo de MUX.

$$F_a(ABCD) = \sum(1, 3, 4, 7, 10, 12, 14)[Mux3, Mux1], [Mux3, Mux2], [Mux1, Mux2]$$

$$F_b(ABCD) = \sum(2, 4, 8, 7, 10, 12, 14, 15)[Mux1, Mux1], [Mux3, Mux3], [Mux2, Mux2]$$

Mux1			
CE	A	B	Out
1	x	x	1
0	0	0	In0
0	0	1	In1
0	1	0	In2
0	1	1	In3

Mux2			
CE	A	B	Out
0	x	x	0
1	0	0	In0
1	0	1	In1
1	1	0	In2
1	1	1	In3

Mux3			
CE	A	B	Out
1	x	x	Z
0	0	0	In0
0	0	1	In1
0	1	0	In2
0	1	1	In3

■ Ejercicio 2

Implementar las siguientes funciones utilizando un multiplexor de 8 a 1.

$$f_a(ABCD) = \sum(1, 3, 4, 6, 7)$$

$$f_b(ABCD) = \Pi(0, 2, 4, 5, 6, 7, 14, 15)$$

$$f_c(ABCD) = \sum(0, 1, 4, 7, 10, 12, 14)$$

■ Ejercicio 3

Implementar las siguientes funciones utilizando un multiplexor de 4 a 1 y compuertas lógicas.

$$F_a(ABCD) = \sum(1, 2, 3, 4, 8, 9, 10, 12, 14)$$

$$F_b(ABCD) = \sum(0, 2, 3, 5, 7, 8, 9, 12, 14, 15)$$

$$F_c(ABCD) = \Pi(1, 3, 4, 6, 7)$$

Se usó el software Falstad para diseñar el circuito (implementación). Dada a su limitación, no se contempla que los MUX cumplan al 100% las tablas de verdad, dado a que no admite el comportamiento de la salida según el pin chip enable.

2. RESPUESTAS

2.1. Ejercicio 1

2.1.1. F_a

■ Tabla de verdad

$F_a(ABCD)$					
m	A	B	C	D	F_a
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	0
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	1
11	1	0	1	1	0
12	1	1	0	0	1
13	1	1	0	1	0
14	1	1	1	0	1
15	1	1	1	1	0

- Señal A usada como conmutador entre dos MUX vía el pin enable.
- Señales B y C van a los selectores del MUX.
- Señal D va a los pines de entrada del MUX.

$F_a(ABCD)$						Entrada al MUX
m	A	B	C	D	F_a	
0	0	0	0	0	0	D
1	0	0	0	1	1	
2	0	0	1	0	0	D
3	0	0	1	1	1	
4	0	1	0	0	1	$\bar{D}$
5	0	1	0	1	0	
6	0	1	1	0	0	D
7	0	1	1	1	1	
8	1	0	0	0	0	D
9	1	0	0	1	0	
10	1	0	1	0	1	$\bar{D}$
11	1	0	1	1	0	
12	1	1	0	0	1	$\bar{D}$
13	1	1	0	1	0	
14	1	1	1	0	1	$\bar{D}$
15	1	1	1	1	0	

■ [Mux3,Mux1]

Para que el Mux3 (minterminos 0,1,2,3,4,5,6,7) refleje las entradas, el pin chip enable (CE) debe estar en 0, por lo tanto irá con la señal A .

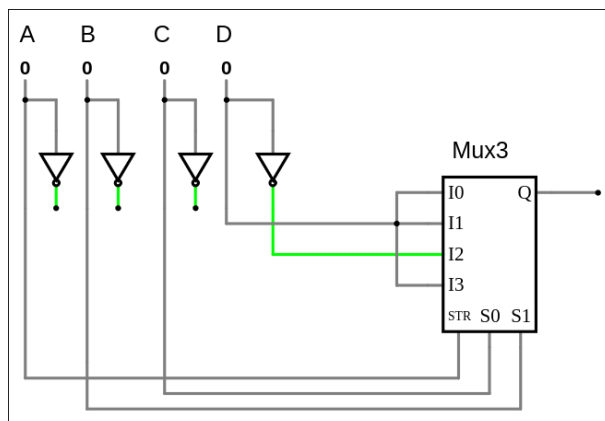


Figura 1: Mux3 Fa Ejercicio 1

Para que el Mux1 (minterminos 8,9,10,11,12,13,14,15) refleje las entradas, el pin chip enable debe estar en 0, recibe la señal A que estará en 1 para continuar la tabla de verdad, entonces en el pin chip enable de Mux1 debe ir un inversor.

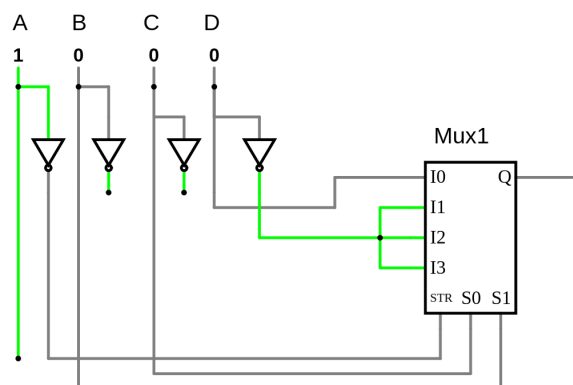


Figura 2: Mux1 Fa Ejercicio 1

Para combinar ambos Mux hay que agregar varios elementos. Cuando Mux3 está reflejando la primera mitad de la tabla de verdad, Mux1 estará en 1 ($A = 0 \rightarrow \text{inversor} \rightarrow 1 \Rightarrow \text{Mux1 en } 1$). Para reflejar los 1 del Mux3, se puede poner una compuerta AND cuya entrada es la salida de los Mux 3 y 1:

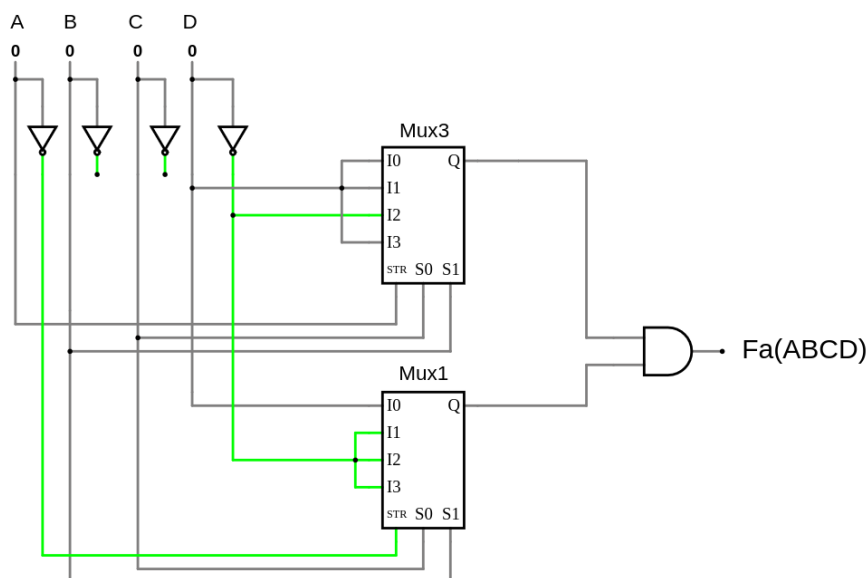


Figura 3: Fa Ejercicio 1 incompleto

Sin embargo, cuando Mux3 tiene un 1 en el chip enable, se pone en alta impedancia su salida. Se puede fijar un 1 lógico con un pull-up. De esta manera, cuando trabaje Mux1, se reflejan los 1 de la función, y cuando esté activo Mux3, va a "ganarle" al pull-up, reflejando lo que se espera: la salida del mux.

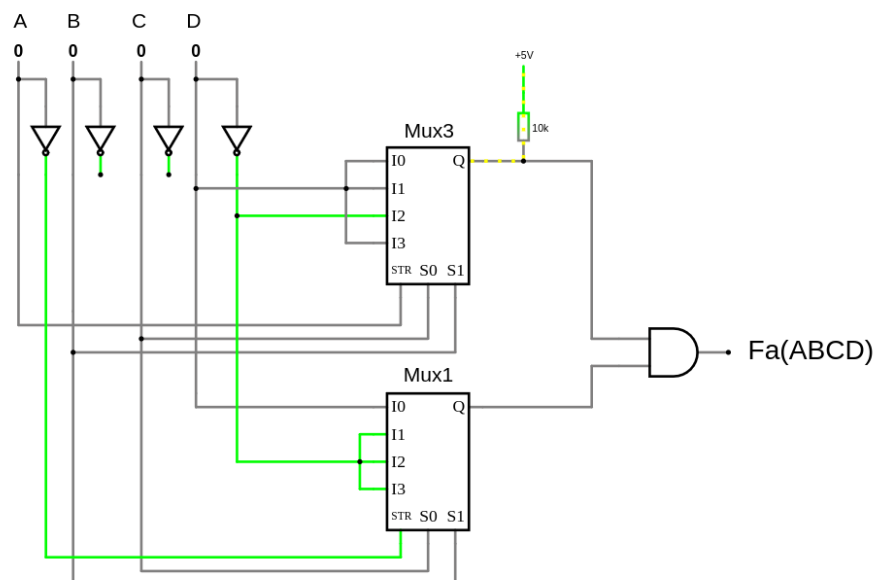


Figura 4: Ejercicio 1 Fa completo

■ [Mux3,Mux2]

Cuando Mux2 tiene un 0 en el pin chip enable, su salida se pone en 0 mientras Mux3 refleja las entradas, por lo que se podría poner una compuerta OR. Cuando el pin chip enable se pone en 1, Mux2 refleja sus entradas y Mux3 pone su salida en alta impedancia, por lo que se podría poner una resistencia pull-down para imponer un 0 lógico y que se reflejen las salidas de Mux2.

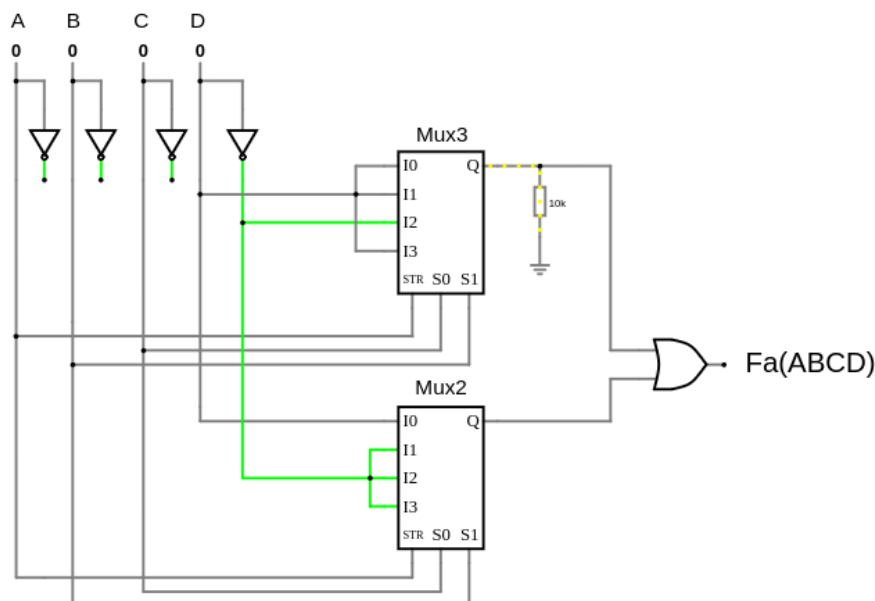


Figura 5: Ejercicio 1 Fa Mux3 y Mux2

■ [Mux1,Mux2]

Cuando el pin enable A está en 0, se activa Mux1 reflejando las entradas y Mux2 queda en 0, por lo que amerita unir las salidas con compuerta OR. Sin embargo, cuando el pin enable es 1, Mux1 se pone en 1 y Mux2 refleja las entradas. Para suprimir los 1 de Mux1 cuando A = 1, se puede introducir una compuerta AND antes del OR, cuya entradas son la salida de Mux1 y el negado de A. Entonces cuando A = 0, en la AND hay un 1 y reflejará los 1 de Mux1, que irán a la compuerta OR mientras Mux2 está en 0 (no habría problema); cuando A = 1, en la compuerta AND habrá un 0 y no pasará el 1 del Mux1 en ese estado, dejando a la compuerta OR solo con los 1 de Mux2.

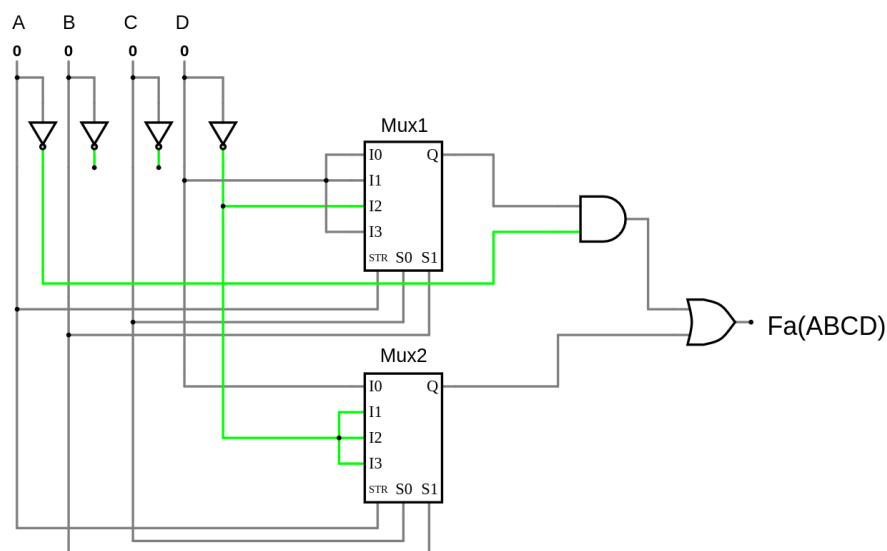


Figura 6: Ejercicio 1 Fa Mux1 y Mux2

2.1.2. F_b

■ Tabla de verdad

$F_b(ABCD)$						
m	A	B	C	D	F_b	Entrada al MUX
0	0	0	0	0	0	0
1	0	0	0	1	0	
2	0	0	1	0	1	$\overline{D}$
3	0	0	1	1	0	
4	0	1	0	0	1	$\overline{D}$
5	0	1	0	1	0	
6	0	1	1	0	0	D
7	0	1	1	1	1	
8	1	0	0	0	1	$\overline{D}$
9	1	0	0	1	0	
10	1	0	1	0	1	$\overline{D}$
11	1	0	1	1	0	
12	1	1	0	0	1	$\overline{D}$
13	1	1	0	1	0	
14	1	1	1	0	1	1
15	1	1	1	1	1	

■ [Mux1,Mux1]

Cuando el pin enable está en 0, trabaja el primer Mux1 reflejando a la salida los minterminos del 0 al 7 de la tabla de verdad de F_b , entras que en simultáneo el segundo Mux1 está en 1. Cuando $A = 1$, se invierten los roles de los Mux pero la lógica es la misma: uno devuelve un 1 mientras que el otro los minterminos de la función, por lo que se podría unir las salidas de los Mux con una AND.

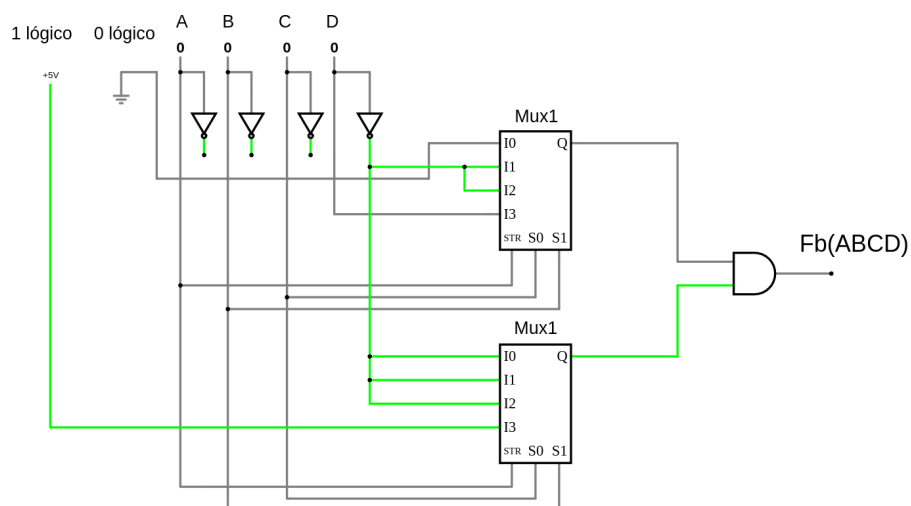


Figura 7: Ejercicio 1 Fb completo Mux1 Mux1

■ [Mux2,Mux2]

Ambas compuertas no pueden tener un mismo valor de la señal chip enable de A, por lo que en Mux2 que gobierna los minitérminos del 0 al 7, tendrá invertido su valor del pin enable para reflejar las entradas mientras el segundo Mux2 (minitérminos del 8 al 15), permanece en 0. Las salidas se unen con una compuerta OR.

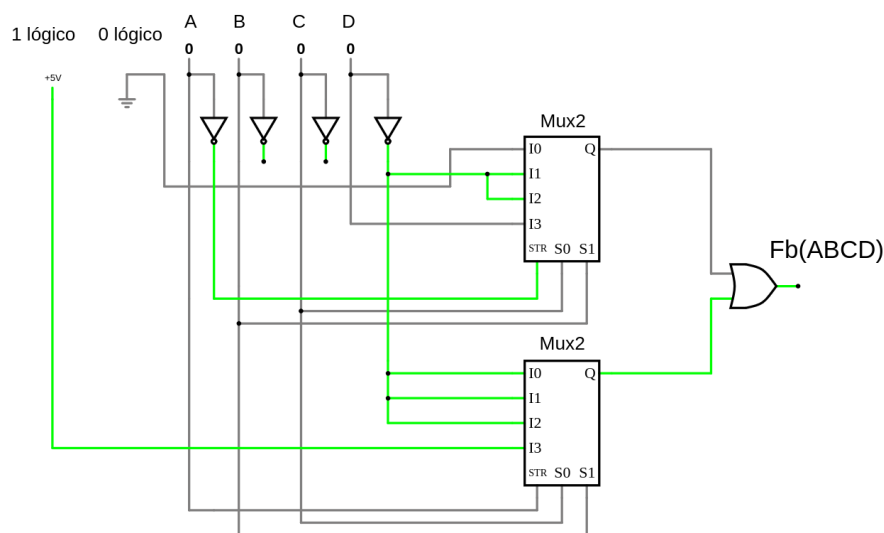


Figura 8: Ejercicio 1 Fb completo Mux2 Mux2

■ [Mux3,Mux3]

El primer Mux3 recibe el 0 del pin enable A mientras que el segundo Mux3, su negado. Entonces uno reflejará las entradas, mientras que el otro tendrá un pull-up dando siempre 1 lógico en alta impedancia. Las salidas se unen a una compuerta AND.

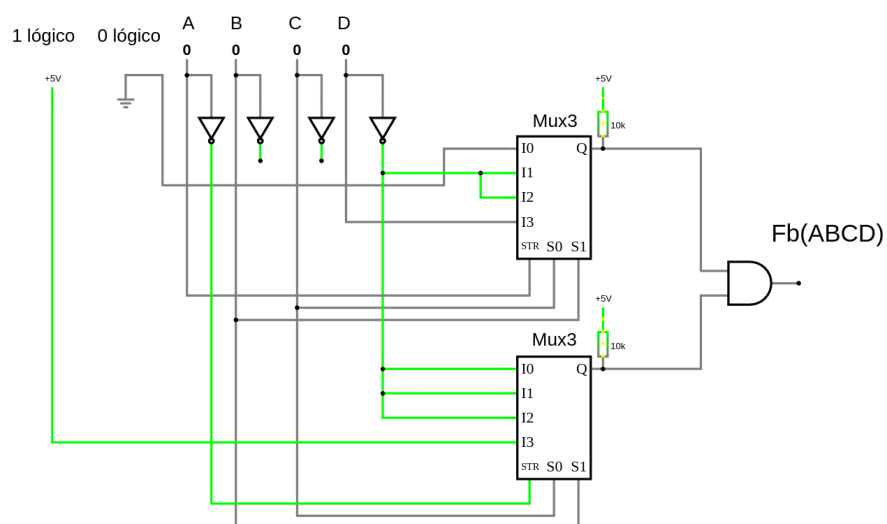


Figura 9: Ejercicio 1 Fb completo Mux3 Mux3

2.2. Ejercicio 2

2.2.1. f_a

■ Tabla de verdad

$f_a(ABCD)$						
m	A	B	C	D	f_a	Entrada al MUX
0	0	0	0	0	0	D
1	0	0	0	1	1	
2	0	0	1	0	0	D
3	0	0	1	1	1	
4	0	1	0	0	1	$\bar{D}$
5	0	1	0	1	0	
6	0	1	1	0	1	1
7	0	1	1	1	1	
8	1	0	0	0	0	0
9	1	0	0	1	0	
10	1	0	1	0	0	0
11	1	0	1	1	0	
12	1	1	0	0	0	0
13	1	1	0	1	0	
14	1	1	1	0	0	0
15	1	1	1	1	0	

■ Implementación

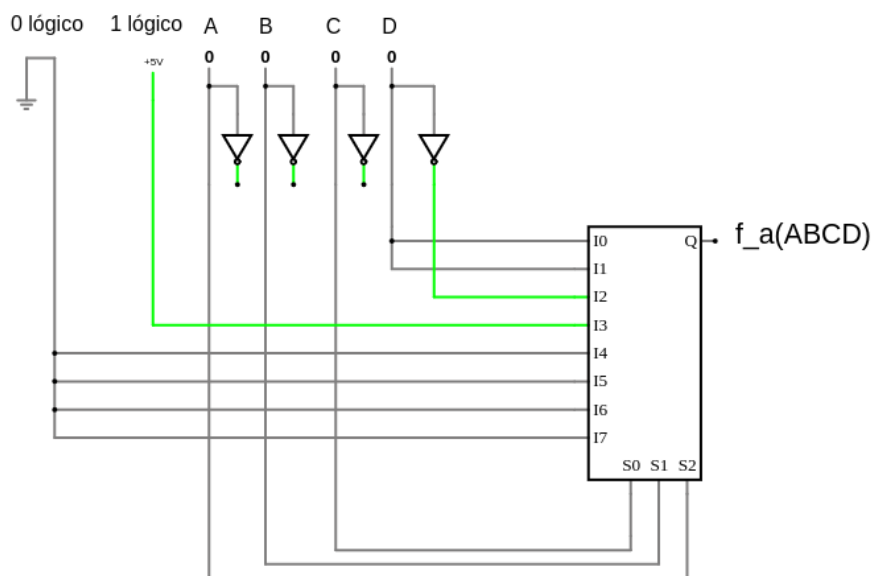


Figura 10: Ejercicio 2 f_a

2.2.2. f_b

■ Tabla de verdad

$f_b(ABCD)$						
m	A	B	C	D	f_b	Entrada al MUX
0	0	0	0	0	0	D
1	0	0	0	1	1	
2	0	0	1	0	0	D
3	0	0	1	1	1	
4	0	1	0	0	0	0
5	0	1	0	1	0	
6	0	1	1	0	0	0
7	0	1	1	1	0	
8	1	0	0	0	1	1
9	1	0	0	1	1	
10	1	0	1	0	1	1
11	1	0	1	1	1	
12	1	1	0	0	1	1
13	1	1	0	1	1	
14	1	1	1	0	0	0
15	1	1	1	1	0	

■ Implementación

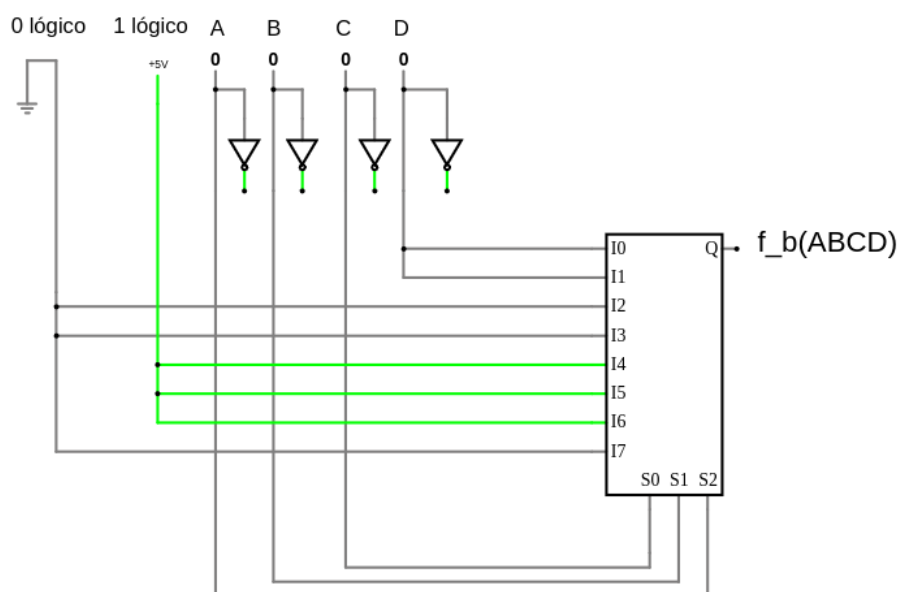


Figura 11: Ejercicio 2 f_b

2.2.3. f_c

■ Tabla de verdad

$f_c(ABCD)$						
m	A	B	C	D	f_c	Entrada al MUX
0	0	0	0	0	1	1
1	0	0	0	1	1	
2	0	0	1	0	0	0
3	0	0	1	1	0	
4	0	1	0	0	1	D
5	0	1	0	1	0	
6	0	1	1	0	0	D
7	0	1	1	1	1	
8	1	0	0	0	0	0
9	1	0	0	1	0	
10	1	0	1	0	1	$\bar{D}$
11	1	0	1	1	0	
12	1	1	0	0	1	$\bar{D}$
13	1	1	0	1	0	
14	1	1	1	0	1	$\bar{D}$
15	1	1	1	1	0	

■ Implementación

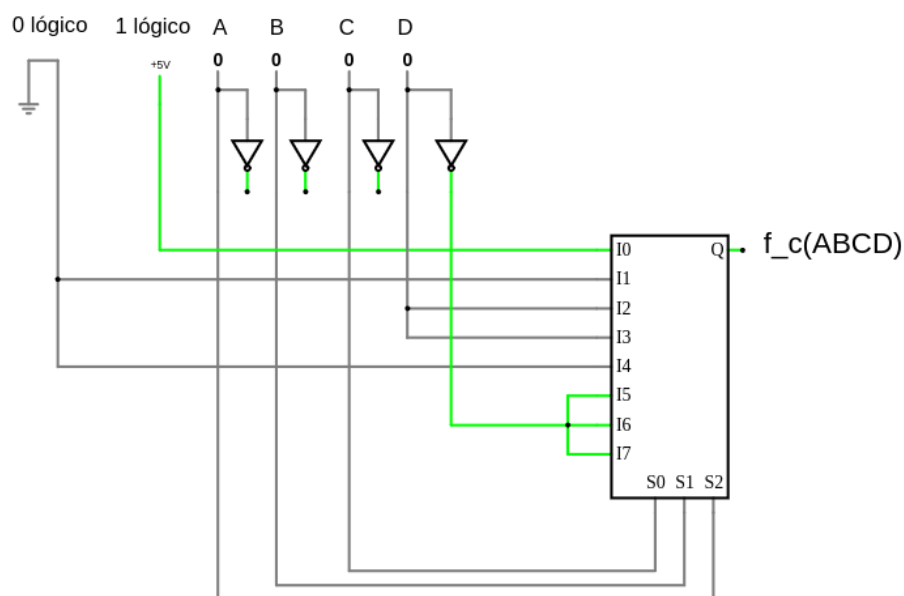


Figura 12: Ejercicio 2 f_c

2.3. Ejercicio 3

La resolución de estos ejercicios estará basado en el Mux2 del ejercicio 1

■ Tablas de verdad

$F_a(ABCD)$						
m	A	B	C	D	F_a	Entrada al MUX
0	0	0	0	0	0	D
1	0	0	0	1	1	
2	0	0	1	0	1	1
3	0	0	1	1	1	
4	0	1	0	0	1	$\bar{D}$
5	0	1	0	1	0	
6	0	1	1	0	0	0
7	0	1	1	1	0	
8	1	0	0	0	1	1
9	1	0	0	1	1	
10	1	0	1	0	1	$\bar{D}$
11	1	0	1	1	0	
12	1	1	0	0	1	$\bar{D}$
13	1	1	0	1	0	
14	1	1	1	0	1	$\bar{D}$
15	1	1	1	1	0	

$F_b(ABCD)$						
m	A	B	C	D	F_b	Entrada al MUX
0	0	0	0	0	1	$\bar{D}$
1	0	0	0	1	0	
2	0	0	1	0	1	1
3	0	0	1	1	1	
4	0	1	0	0	0	D
5	0	1	0	1	1	
6	0	1	1	0	0	D
7	0	1	1	1	1	
8	1	0	0	0	1	1
9	1	0	0	1	1	
10	1	0	1	0	0	0
11	1	0	1	1	0	
12	1	1	0	0	1	$\bar{D}$
13	1	1	0	1	0	
14	1	1	1	0	1	1
15	1	1	1	1	1	

$F_c(ABCD)$						
m	A	B	C	D	F_c	Entrada al MUX
0	0	0	0	0	1	$\bar{D}$
1	0	0	0	1	0	
2	0	0	1	0	1	$\bar{D}$
3	0	0	1	1	0	
4	0	1	0	0	0	D
5	0	1	0	1	1	
6	0	1	1	0	0	0
7	0	1	1	1	0	
8	1	0	0	0	1	1
9	1	0	0	1	1	
10	1	0	1	0	1	1
11	1	0	1	1	1	
12	1	1	0	0	1	1
13	1	1	0	1	1	
14	1	1	1	0	1	1
15	1	1	1	1	1	

■ Implementación

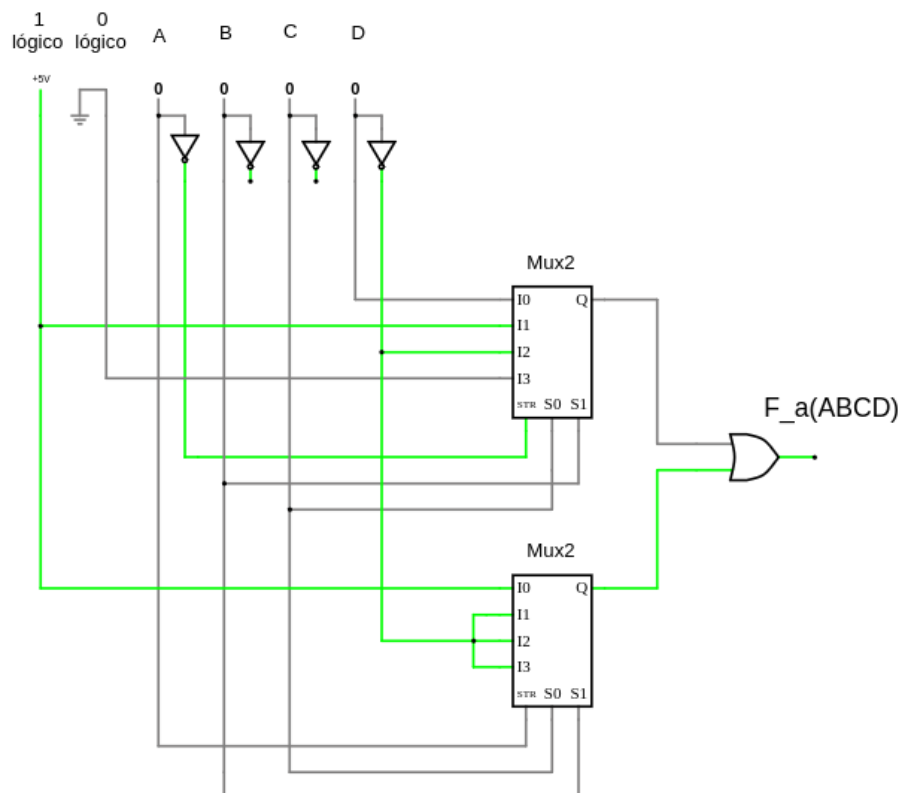


Figura 13: Ejercicio 3 F_a

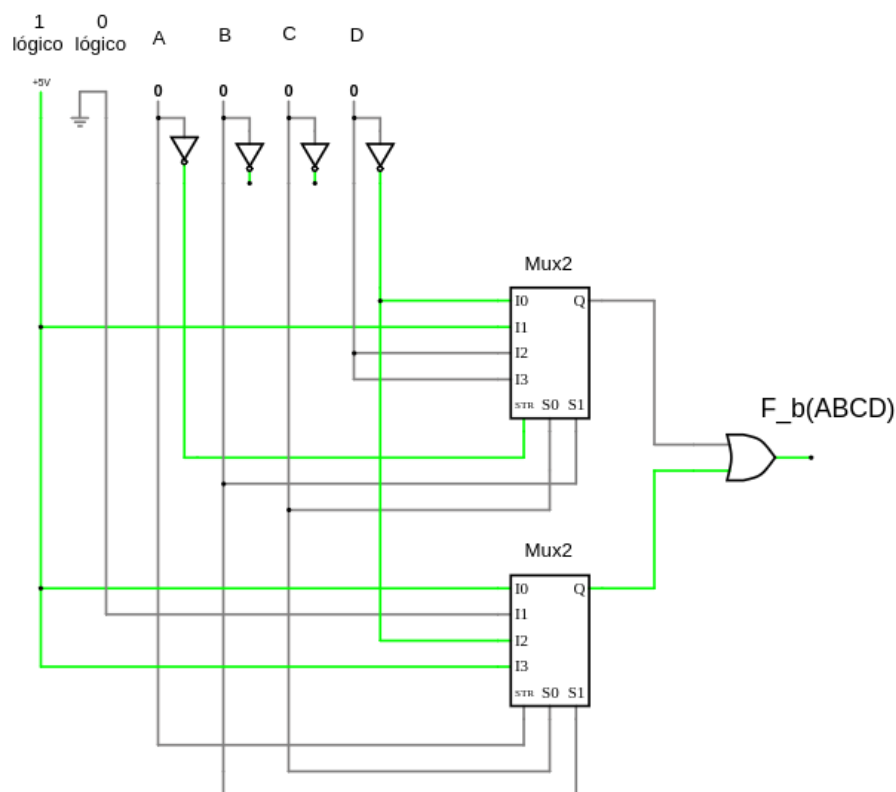


Figura 14: Ejercicio 3 F_b

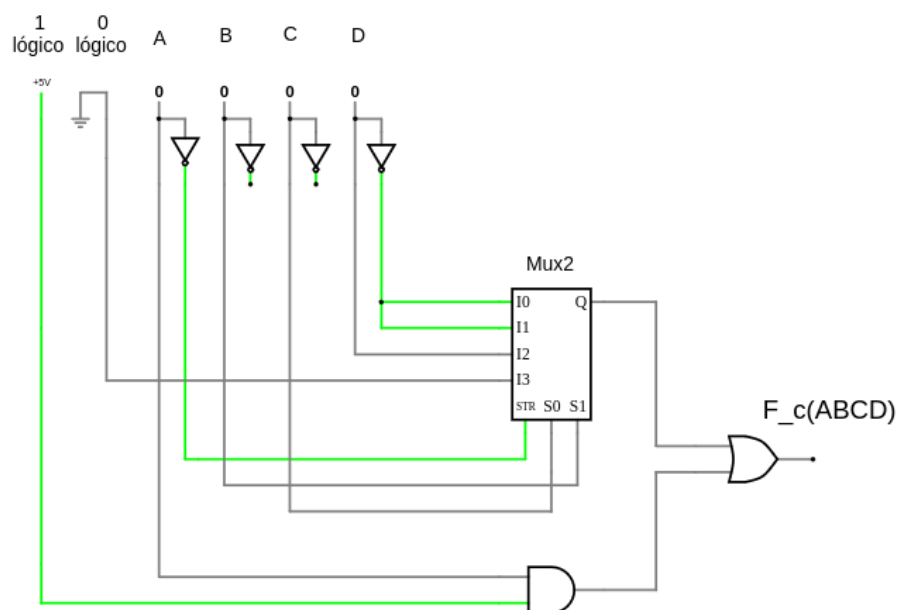


Figura 15: Ejercicio 3 F_c